



Servlet + Request + Response

- 1 Servlet的生命周期
- 2 ServletContext对象
- 3 Response对象
- 4 Request对象

- 1 **Servlet的生命周期**
- 2 ServletContext对象
- 3 Response对象
- 4 Request对象

Servlet的生命周期

□ 什么是生命周期？

- 生命周期是指一个对象从被创建到被销毁的过程。

□ Servlet的生命周期？

- Servlet对象从创建到销毁的过程。
- Servlet中有init, service, destroy方法，这几个方法称为是Servlet生命周期相关的方法。
- Servlet在第一次被访问的时候会被实例化，调用init方法(只执行一次)；
- 任何一次从前端发送来的请求，会调用Servlet中的service方法(执行多次)；
- 当Servlet从服务器中移除或者服务器关闭时，Servlet对象被销毁的destroy方法会执行。

注：只要服务器不关闭，Servlet对象一直都存在！

Servlet的生命周期

□Servlet的启动时加载

- 一般来说，Servlet对象在第一次被访问的时候会被实例化，调用init方法。
- 如何提前至服务器启动时就实例化Servlet对象呢？

(1)为什么要提前？

有时会在init方法中做了一些比较耗时的操作，如：加载数据库连接等资源，此时用户要进行等待。

(2)如何提前？（在tomcat服务器启动时加载Servlet）在web.xml进行配置，

`<load-on-startup>2</load-on-startup>`

```
<?xml version="1.0" encoding="UTF-8"?>
<servlet>
  <description></description>
  <display-name>MyServlet_1</display-name>
  <servlet-name>MyServlet_1</servlet-name>
  <servlet-class>it.hy.hj.servletDemo1.MyServlet_1</servlet-class>
  <!--Servlet启动时加载-->
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>MyServlet_1</servlet-name>
  <url-pattern>/MyServlet_1</url-pattern>
</servlet-mapping>
```

- 1 Servlet的生命周期
- 2 ServletContext对象**
- 3 Response对象
- 4 Request对象

□ServletContext对象

– 什么是ServletContext对象？

(1)Servlet的上下文对象，一个web项目只有一个ServletContext对象。

(2)服务器启动时，为每个web项目创建一个单独的ServletContext对象。

– ServletContext的作用？

(1)获取Web项目的信息；

(2)读取Web项目下的文件；

(3)作为域对象存取数据。

– ServletContext对象的获取？

在HttpServlet类的方法中，使用this.getServletContext()获取该对象。

ServletContext对象

□ ServletContext对象作用——获取Web项目信息

— 获取Web项目的名称

String	<code>getContextPath()</code> Returns the context path of the web application.
--------	---

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {  
    // TODO Auto-generated method stub  
    ServletContext sc = this.getServletContext();  
    String webProjName = sc.getContextPath();  
    response.getWriter().write("项目名称是: " + webProjName);  
}
```

运行结果： ?????: /MyWebProj01

出现中文乱码！！

□ServletContext对象作用——读取Web项目下的文件

– 读取Web项目下的文件

<u>String</u>	<u>getRealPath</u> (<u>String</u> path) Returns a String containing the real path for a given virtual path.
---------------	---

```
ServletContext sc = this.getServletContext();
String realPath = sc.getRealPath("/srcFileFolder/1.txt");
BufferedReader br = new BufferedReader(new FileReader(realPath));
String strLine = null;
while ((strLine = br.readLine()) != null) {
    response.getWriter().write(strLine + "\n");
}
```

注：路径的写法，从Webapps下的目录开始！

红色方框中的代码可以实现对文本文件的读取，并输出到浏览器的页面中

运行结果：

```
connURL: jdbc:mysql://localhost:3306/web_test3
username: root
password: abc
```

□ ServletContext对象作用——作为域对象存取数据

– 什么是域对象？

域对象是指一个对象可以存储一定范围内的所有数据，通过该对象就能获取和存储数据。域指的是一定的范围。

– JavaWeb中一共有4种域对象（作用范围从小到大）

PageContext域、HttpServletRequest域、session域、ServletContext域

– ServletContext域对象的常用方法（获取、移除、存入数据）

<u>Object</u>	<u>getAttribute</u> (<u>String</u> name) Returns the servlet container attribute with the given name, or null if there is no attribute by that name.
<u>void</u>	<u>removeAttribute</u> (<u>String</u> name) Removes the attribute with the given name from the servlet context.
<u>void</u>	<u>setAttribute</u> (<u>String</u> name, <u>Object</u> object) Binds an object to a given attribute name in this servlet context.

注：一般是将后端运行的结果数据存放在域对象中，由response对象返回到前端

□ ServletContext对象作用——作为域对象存取数据

- ServletContext域对象的作用范围

创建：Web项目所在的服务器启动时；

销毁：服务器关闭或Web项目被移出服务器；

个数：一个Web项目有且仅有一个ServletContext；

作用范围：整个Web项目。

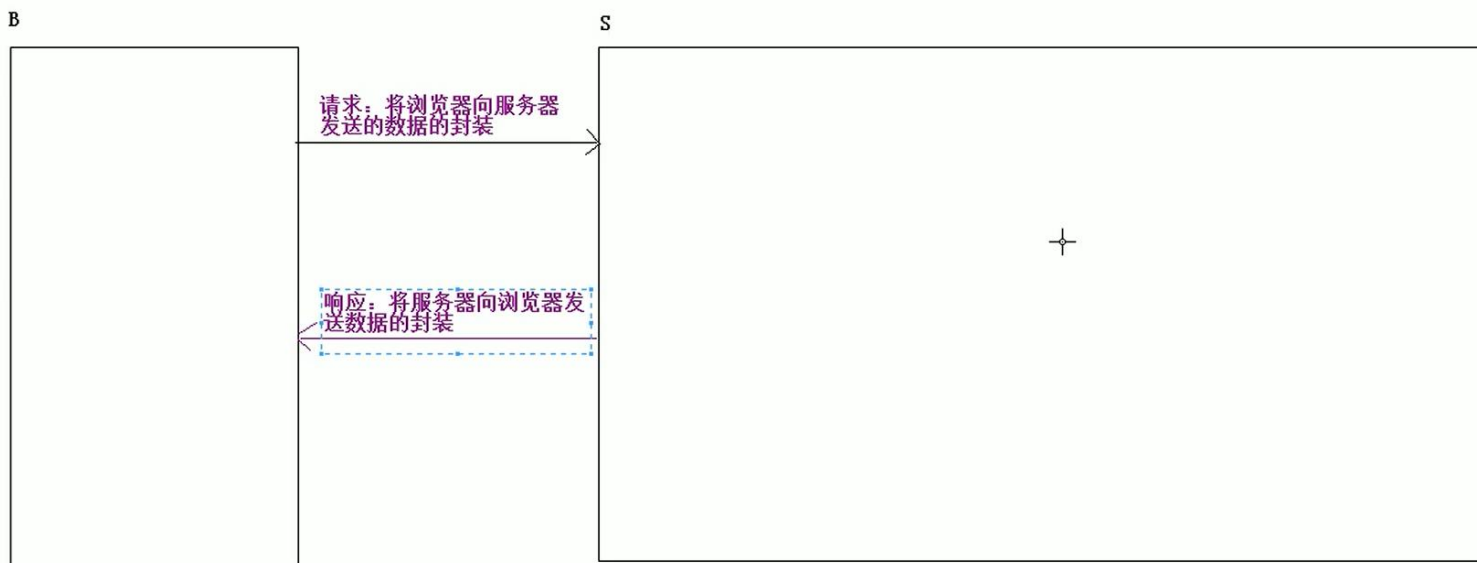
- 1 Servlet的生命周期
- 2 ServletContext对象
- 3 **Response对象**
- 4 Request对象

□Response对象概述

– 什么是Response?

对于B/S结构的软件，从浏览器输入一个地址访问服务器（这个过程称为是请求，Request）。服务器接收到请求，需要进行处理，处理以后需要将处理结果显示回浏览器端（这个过程称为是响应，Response）。

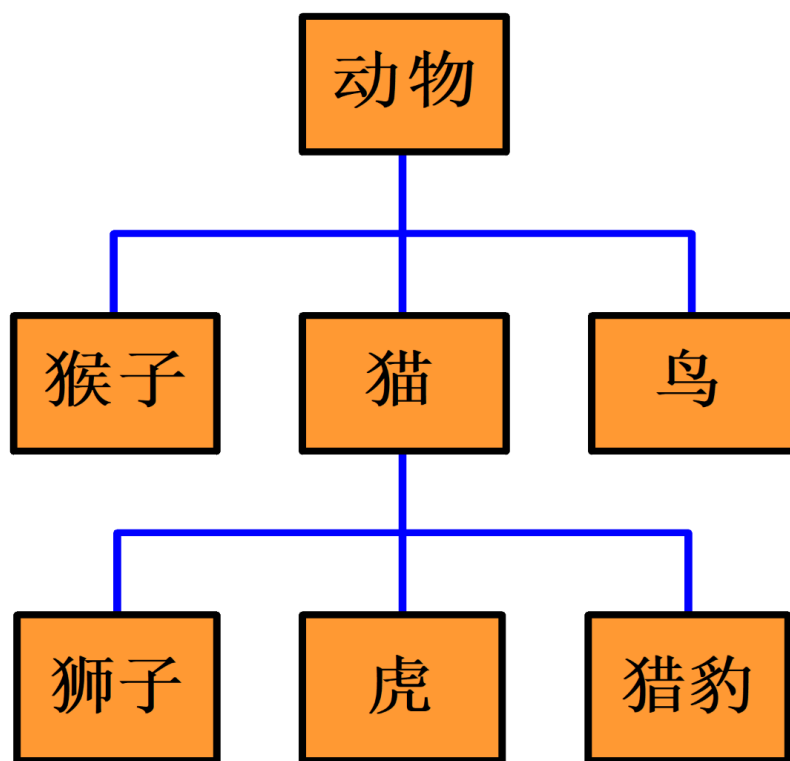
服务器的请求和响应



知识点：继承特性

□ 面向对象的继承特性

— 什么是继承？



继承是描述类之间的一种关系，下一级别的类拥有上一级别的
共性特征，还有自己的个性特征。

□ 面向对象的继承特性

— 类的继承

- 保持已有类的共性而构造具有个性特征的新类的过程称为继承。
- 被继承的类称为：父类；构造的新类：子类。
- 子类的成员 = 父类的成员（属性与方法）+ 自身的成员

例：类A有属性f1, f2, f3, 有方法func_1(), func_2();

类a继承类A, 且类a有自身的属性f4, 自身的方法func_3();

类a所具有的属性有f1, f2, f3, f4, 方法有func_1(), func_2(),

func_3()

Response对象

□Response对象的常用方法

ServletResponse

|

HttpServletResponse

– Response关于响应行的方法

void	<u>setStatus</u> (int sc) Sets the status code for this response.
------	--

设置状态码：200 正确，302 重定向，304 查找本地缓存，404 请求资源不存在，
500 服务器内部错误（HSR）

– Response关于响应头的方法

void	<u>setHeader</u> (String name, String value) Sets a response header with the given name and value.
------	---

设置响应头，如：文档编码、文档类型、文档定时刷新等。（HSR）

– Response关于响应体的方法

<u>ServletOutputStream</u>	<u>getOutputStream</u> () Returns a <u>ServletOutputStream</u> suitable for writing binary data in the response.
<u>PrintWriter</u>	<u>getWriter</u> () Returns a <u>PrintWriter</u> object that can send character text to the client.

获取输出流对象，向html页面输出内容（SR）

Response对象

□Response对象的常用方法

– Response对象的其它方法

void	<code>sendRedirect</code> (<code>String</code> location)
------	---

Sends a temporary redirect response to the client using the specified redirect location URL.

重定向（将网络请求重新定向，转到其它位置，HSR）

void	<code>setContentType</code> (<code>String</code> type)
------	---

Sets the content type of the response being sent to the client, if the response has not been committed yet.

设置浏览器打开页面时候采用的字符集编码（SR）

void	<code>setCharacterEncoding</code> (<code>String</code> charset)
------	--

Sets the character encoding (MIME charset) of the response being sent to the client, for example, to UTF-8.

设置响应字符流的缓冲区字符集编码（SR）

□Response响应的中文乱码问题

– Response向页面响应中文

ServletOutputStream	getOutputStream() Returns a ServletOutputStream suitable for writing binary data in the response.
PrintWriter	getWriter() Returns a PrintWriter object that can send character text to the client.

获取输出流对象，向html页面输出内容；其中，ServletOutputStream是字节流；PrintWriter是字符流。

(1) 使用字节流方式响应中文

```
ServletOutputStream outputStream = response.getOutputStream();  
outputStream.write("中文".getBytes());
```

使用上述代码向页面输出中文是否会有乱码？ **不一定**

原因分析：其实这个乱码的产生与中文转成字节数组及浏览器打开时采用的默认字符集有关。两者一致就不会产生乱码！

□Response响应的中文乱码问题

– Response向页面响应中文

解决：将中文转成字节数组和浏览器打开时默认采用的字符集设置一致即可

```
// 使用字节流的方式输出中文：  
ServletOutputStream outputStream = response.getOutputStream();  
// 设置浏览器默认打开的时候采用的字符集  
response.setHeader("Content-Type", "text/html;charset=UTF-8"); 设置响应文档的编码  
// 设置中文转成字节数组字符集编码  
outputStream.write("中文".getBytes("UTF-8")); 设置转为字节数组的编码
```

(2) 使用字符流方式响应中文

```
//使用字符流  
response.getWriter().write("互联网金融");|
```

使用上述代码向页面输出中文是否会有乱码？ **一定乱码**

原因分析：字符流是有缓冲区的，response获得字符流，其默认的缓冲区编码是ISO-8859-1。这个字符集不支持中文的。

□Response响应的中文乱码问题

– Response向页面响应中文

解决：设置response获得字符流缓冲区的编码和浏览器默认打开时采用的字符集一致即可。

//设置浏览器打开页面时候采用的字符集编码

`response.setContentType("text/html;charset=utf-8");` 设置响应文档的编码

//设置响应字符流的缓冲区字符集编码

`response.setCharacterEncoding("utf-8");`

`response.getWriter().write("互联网金融");` 设置字符流缓冲区的编码

- 1 Servlet的生命周期
- 2 ServletContext对象
- 3 Response对象
- 4 **Request对象**

Request对象

Request对象概述

– 什么是Request?

对于B/S架构软件，从浏览器向服务器发送请求，该请求往往包含了一些数据，将这些内容进行封装就封装成了一个请求对象（Request对象）。

Request对象的常用方法

– Request关于获取客户端机器信息的方法

ServletRequest

|

HttpServletRequest

String getMethod()

Returns the name of the HTTP method with which this request was made, for example, GET, POST, or PUT.

获得请求方式（HSR）

String getRemoteAddr()

Returns the Internet Protocol (IP) address of the client or last proxy that sent the request.

获得客户端机器的IP地址（SR）

Request对象

Request对象的常用方法

Request关于请求头的方法

<u>String</u>	<u>getHeader</u> (<u>String</u> name)	Returns the value of the specified request header as a String.
---------------	--	--

获取请求头的信息，如：Cookie。（HSR）

Request关于请求参数的方法

<u>String</u>	<u>getParameter</u> (<u>String</u> name)	Returns the value of a request parameter as a String, or null if the parameter does not exist.
---------------	---	--

<u>String</u> []	<u>getParameterValues</u> (<u>String</u> name)	Returns an array of String objects containing all of the values the given request parameter has, or null
------------------	---	--

获取提交的参数，如表单中的用户名、密码（一个值）、爱好（多个值）等！（SR）

Request作为域对象存取数据的方法（获取、移除、存入数据）

<u>Object</u>	<u>getAttribute</u> (<u>String</u> name)	Returns the servlet container attribute with the given name, or null if there is no attribute by that name.
---------------	---	---

void	<u>removeAttribute</u> (<u>String</u> name)	Removes the attribute with the given name from the servlet context.
------	--	---

void	<u>setAttribute</u> (<u>String</u> name, <u>Object</u> object)	Binds an object to a given attribute name in this servlet context.
------	---	--

Request对象

□Request作为域对象作用范围

Request域对象其实就是从客户端浏览器向服务器发送的一次请求信息的封装。

那么实质上向Request中所保存的数据有效期也是一次请求范围。

一次请求范围：从客户端浏览器向服务器发送一次请求，服务器针对这次请求对浏览器作出响应。当服务器作出响应之后，请求对象就销毁了，保存在其中的数据就无效了。

– ServletContext域对象 VS Request域对象

域对象	生命周期	作用范围
ServletContext	首次请求或服务器启动时被创建； 项目被移除服务器或服务 器关闭；	整个Web项目
Request	浏览器发送请求时被创建； 服务器做出响应后被销毁。	一次请求

□Request对象的常用方法

– Request对象的其它方法

```
void | setCharacterEncoding(String charset)  
      Sets the character encoding (MIME charset) of the response being sent to the client, for example, to UTF-8.
```

设置request对象缓冲区字符集编码（SR）

Request对象

Request获取表单提交的数据

1. 编写静态表单

```
<h1>request接收表单参数</h1>
<form action="/web01/RequestDemo2" method="get">
    用户名:<input type="text" name="username"/><br/>
    密码:<input type="password" name="password"/><br/>
    性别:<input type="radio" name="sex" value="man"/>男<input type="radio" name="sex" value="woman"/>女<br/>
    籍贯:<select name="city">
        <option value="beijing">北京市</option>
        <option value="shanghai">上海市</option>
        <option value="shenzhen">深圳市</option>
    </select><br/>
    爱好:<input type="checkbox" name="hobby" value="basketball"/>篮球
        <input type="checkbox" name="hobby" value="football"/>足球
        <input type="checkbox" name="hobby" value="volleyball"/>排球<br/>
    自我介绍:<textarea name="info" rows="3" cols="8"></textarea>
    <input type="submit" value="提交">
</form>
```

2. 获取表单提交的数据

```
// 接收用户名和密码:
String username = request.getParameter("username");
String password = request.getParameter("password");
// 接收性别和籍贯:
String sex = request.getParameter("sex");
String city = request.getParameter("city");
// 接收爱好:
String[] hobby = request.getParameterValues("hobby");
// 接收自我介绍
String info = request.getParameter("info");
```

```
System.out.println("用户名:"+username);
System.out.println("密码:"+password);
System.out.println("性别:"+sex);
System.out.println("籍贯:"+city);
System.out.println("爱好:"+Arrays.toString(hobby));
System.out.println("自我介绍:"+info);
```

Request接收表单参数的中文乱码问题

- Post方式接收中文：一定会产生乱码

原因分析：Post方式提交的数据在请求体中，request对象接收到数据后会将其放入到缓冲区中，该缓冲区编码默认是ISO-8859-1，不支持中文；

解决：设置request缓冲区的字符集编码

```
request.setCharacterEncoding("UTF-8");  
String name = request.getParameter("name");  
System.out.println("姓名:"+name);
```

- Get方式接收中文：一定会产生乱码

原因分析：Get方式提交的数据在请求行的地址栏中，地址栏上的文本以及经过一次URL编码，再放入到request缓冲区中；

解决：将存入到request缓冲区中的内容用ISO-8859-1的方式获取，然后用utf-8进行解码。

Request接收表单参数的中文乱码问题

- Get方式接收中文：一定会产生乱码

原因分析：Get方式提交的数据在请求行的地址栏中，地址栏上的文本以及经过一次URL编码，再放入到request缓冲区中；

解决：将存入到request缓冲区中的内容用ISO-8859-1的方式获取，然后用utf-8进行解码。

```
String name = request.getParameter("name");  
/*String encode = URLEncoder.encode(name, "ISO-8859-1");  
String decode = URLDecoder.decode(encode, "UTF-8");  
System.out.println("姓名:"+decode);*/  
String value = new String(name.getBytes("ISO-8859-1"), "UTF-8");  
System.out.println("姓名:"+value);
```

谢谢!

Thanks for your attention!